

第 3 课 · RAG、证据链与可信回答

从观察后检索到带边界的结论，理解引用能保证什么

从概念、项目代码到运行证据的连续阅读材料。

目录

第 3 课 • RAG、证据链与可信回答	4
3.1 本章解决的问题	4
3.2 贯穿案例：让知识解释观察	4
先有时间线，再找处理方法	4
3.3 观察后检索：查询也是调查结果	5
把模糊问题改成可区分的线索	5
当前 V2 工具的直接观测	6
3.4 知识与答案的资产边界	7
3.5 检索技术：先建立直觉	8
词法、向量与混合各解决什么	8
重排放在哪里	8
3.6 读当前 V2 检索器	9
片段长度与定位	10
3.7 V1 基线：弱检索器怎样拖累融合	10
数据和计算方法	10
为什么“只用排名”仍可能错	11
3.8 证据对象：不把摘录变成无来源文字	12
稳定 ID 与去重的实际边界	13
3.9 完整性、截断与假设关系	13
没有看到不等于没有发生	13
支持、反驳与相关不是同一种关系	14
3.10 主张—证据校验：引用合法仍有边界	14
把重要结论拆成可核查的句子	14
引用存在但结论仍错误的反例	15
3.11 证据不足：带事实与缺口结束	16
3.12 运行证据与反例检查	16

两次可复跑调查16

阅读时主动寻找这些反例17

3.13 本章总结与课后阅读17

一条建议的源码阅读路线17

课后模仿方向与思考18

第 3 课 · RAG、证据链与可信回答

从观察后检索到带边界的结论，理解引用能保证什么

3.1 本章解决的问题

一份回答引用了企业文档，就一定可信吗？上一章让调查能够停下来，本章追问它凭什么得出结论。模型可以了解 CUDA、通信超时等通用概念，却没有看过本次任务的四个 worker，也不知道企业当前的处理规范。应用必须把本次观察和有关知识送到模型面前，并保留每个重要判断所依赖的来源。

RAG 是 Retrieval-Augmented Generation 的缩写，可以译为检索增强生成。应用先从外部资料找出相关内容，再将它们作为上下文提供给模型，使回答有机会依据当前资料展开。检索不更新模型权重，也不让检索到的文字自动成为事实。本项目把这条链延伸到调查过程：先读任务、日志和指标，再检索 Runbook，最后检查报告引用。

读完本章，应能区分四件事：检索结果相关、观察来源明确、报告引用合法、主张确实受到证据支持。它们有关联，但不能互相替代。我们仍围绕 case-gpu-assert 的 job-v2-101 阅读，随后用 case-insufficient-evidence 观察资料不足时怎样结束调查。所有代码都已写好，正文给出阅读路线和可复跑命令。

本章 V2 数字采集于 2026-10-08，基于 64d980e 的当前代码、教学案例和人工编写回放（authored replay）。回放中的根因、置信度和 token usage 是预置响应内容。它们检验集成流程，不是在线模型准确率或真实生产事故统计。

章节关联：第 1 课 · 消息、工具调用和 authored replay 的输入匹配边界见第 1 课。

3.2 贯穿案例：让知识解释观察

先有时间线，再找处理方法

case-gpu-assert 的关键观察是：rank 2 在 01:01:00 UTC 报 Indexing.cu 索引断言和 device-side assert，其他 rank 到 01:11:00 前后才报 ALLREDUCE watchdog timeout。任务的 collective_timeout_s 为 600。相隔十分钟的时间线使“本地进程先失败、其余参与者随后等待超时”成为有根据的候选解释。若只把最后一条 timeout 交给模型，网络故障和上游进程退出都可能被泛泛地列出来。

此时需要 Runbook 回答的不是“这个案例的正确标签是什么”，而是“出现这种索引断言时，怎样核实先后关系，怎样区分显存耗尽，还缺哪些信息”。日志给出此次发生的事情，Runbook 给出一般处理方法。两类资料相遇，才能把常识变成针对这次任务的调查建议。

来源	能提供什么	不能单独证明什么
本次各 rank 日志	断言、超时、退出的原文与时间	未采集到的样本值或异步 kernel 起点

来源	能提供什么	不能单独证明什么
本次指标	采样区间内利用率、内存与容量	采样间隔内从未出现瞬时峰值
Runbook	解释模式、辨别步骤、取证建议	本次任务一定符合文档所述原因
模型报告	对观察的判断、未知项与建议	因语言流畅而自动成为可靠事实

GPU assert Runbook 提醒：CUDA 异步执行会让 Python 堆栈指向较晚操作；索引断言可提示非法索引，却不能单独确定哪条样本、预处理步骤或 kernel 引入了问题。报告因此可以判定首个已观察故障是 GPU device assertion，同时保留未知项：The offending input and precise kernel origin remain unproven.更具体的修复方向需要原始输入、同步执行跟踪或隔离复现。

源码定位：data/v2/cases/case-gpu-assert.json

源码定位：data/v2/knowledge/runbooks/gpu-assert.md

3.3 观察后检索：查询也是调查结果

把模糊问题改成可区分的线索

“为什么失败了”描述用户目标，却几乎没有检索信号。观察后检索的顺序是：读取事实、提取具体错误或指标形态、构造查询、再看知识是否帮助区分假设。在 GPU assert 案例中，Indexing.cu、device-side assert、embedding lookup比 job-v2-101 更能定位有关知识。任务 ID 适合读取本次数据，通常不适合查一般性的处理手册。

查询不是越长越好。整段日志中的时间戳、路径、普通 INFO 词和重复 warning 可能稀释有区分力的错误。可以保留稳定错误串和操作名称，必要时再增加上下文；同时保存构造查询所依据的证据，避免模型把猜测悄悄塞进查询。例如看到 timeout 后就查询“坏网卡怎样修复”，会把尚未证明的假设预先写成答案。



图 3-1 · 检索参与调查；模型根据已读到的线索决定查询

当前实现没有硬编码规则强迫所有在线请求先调用 logs 再调用 search_runbooks。工具说明引导模型使用已观察的信号，默认回放也示范这个次序，实际在线顺序仍由模型选择。因此观察后检索是一项设计原则和待评测行为，不是只靠写一句提示就已经实现的强制保障。

当前 V2 工具的直接观测

query	返回结果	读数的边界
为什么失败了	0 个匹配	英文 Runbook 与此中文查询没有词项重叠
Indexing.cu device-side assert embedding lookup	gpu-assert; score=4; 1 个匹配	观察中的稳定词项提供定位信号
launcher exit code 1 missing worker stderr	两篇都 score=5; collective-timeout 在前	同分由文件名排序，第一名并不更可信

这些数值由 build_runbook_tool().fn(query=..., top_k=3) 直接返回，与模型无关。第三行值得保留：两篇手册均讨论先前 worker 日志不足，词项匹配不能自动完成根因区分。检索为空也不能直接判定无知识可用；可能只是语言、术语或索引范围不匹配。后续可尝试英文错误原文或不同查询，但不应拿无关资料填满回答。

课后可复跑示例：输出 1 与 `[('runbook:gpu-assert', 4)]`

```
from ailab_ops.tools.runbooks import build_runbook_tool
tool = build_runbook_tool()
result = tool.fn(
    query="Indexing.cu device-side assert embedding lookup",
    top_k=3,
)
print(result.data["total_matches"])
print([(m["doc_id"], m["score"])
        for m in result.data["matches"]])
```

源码定位：`src/ailab_ops/tools/runbooks.py · build_runbook_tool`

3.4 知识与答案的资产边界

Runbook 是给调查者看的知识。评测标签是对结果评分用的答案和条件。若把某案例的 `expected_root_cause` 塞进工具结果，模型只需复述标签就能拿高分；这衡量的是答案泄漏，不是调查能力。V2 将这两类资料放在不同目录，让默认案例工具只加载观察资产和独立编写的 Markdown 手册。

资产	当前路径	消费角色
案例观察	<code>data/v2/cases/</code>	CaseWorld 与 <code>get_case_*</code> 工具
Runbook	<code>data/v2/knowledge/runbooks/</code>	<code>search_runbooks</code> 返回可引用资料
评测标签	<code>data/v2/evals/labels.jsonl</code>	<code>evals runner</code> 与 <code>scorer</code> ；不进入案例工具
人工编写回放	<code>data/v2/replays/investigations.jsonl</code>	<code>replay gateway</code> 的精确请求/响应记录

独立维护不等于主题互不相关。标签、案例和手册可以都讨论 GPU assert；要避免的是由同一真相模板自动产生可见知识与隐藏答案，或者运行时把标签交给模型。当前 labels 仍由人编写，样本量有限，将来应加入新事故、不同说法和诱导性线索，检验模型能否处理未预置的情况。

源码摘录：工具只遍历指定知识目录的 Markdown

```
directory = Path(root) if root is not None else PROJECT_ROOT / "data/v2/knowledge/runbooks"
documents = []
for path in sorted(directory.glob("*.md")):
    text = path.read_text(encoding="utf-8")
    title = text.splitlines()[0].lstrip("# ") if text else path.stem
    documents.append((path.name, title, text, _terms(title), _terms(text)))
```

源码定位：`src/ailab_ops/tools/runbooks.py · build_runbook_tool`

CaseWorld 包含 jobs、logs、metrics、nodes、incidents、actions 和 telemetry，没有 `root_cause` 标签字段。`load_case` 读取指定 case JSON；默认工具注册另行加入 `build_runbook_tool`。评测侧才读取 labels。这条导入和数据路径边界比“请勿看答案”的提示更可靠。离线回放本身预置诊断，必须单独标明：其结论不能成为在线模型未知案例泛化的证明。

源码定位：src/ailab_ops/cases/loader.py · CaseWorld

源码定位：src/ailab_ops/tools/cases.py · build_case_registry

源码定位：src/ailab_ops/evals/runner.py · load_eval_labels

章节关联：第 5 课 · 独立标签怎样参与分层评测，以及引用分数的限制，见第 5 课。

3.5 检索技术：先建立直觉

词法、向量与混合各解决什么

词法检索依据词项重叠：查询出现 Indexing.cu，文档也出现这个名字，便有匹配信号。稀有错误串往往很有用，而通用的 error 区分力很弱。BM25 用词频、词项稀有度和文档长度组织这类信号；字段加权还可使标题、错误签名比长正文更重要。分词与规范化决定 srcIndex、snake_case 等技术文本能否匹配。

向量检索先将查询和文档编码为一组数字，再根据相似度寻找候选。经过训练的 embedding 模型有机会把“显存不够”和“GPU allocation failed”这样的不同表述放到邻近位置，但语义接近不等于事实相同：型号、版本、错误代码和否定条件仍可能决定适用性。向量是检索表示，不是“知识被证明”的数值。

术语 · embedding

文本的数值表示。查询与文档应使用相容的编码方式。模型版本、维度和预处理改变后，通常要重新构建并评测索引；只更换查询编码器会使相似度失去原有意义。

混合检索从多个检索器取得候选，再融合结果。加权分数需要处理量纲；词法得分 12 与余弦相似度 0.7 不能直接看作前者更可信。RRF（Reciprocal Rank Fusion，倒数排名融合）只使用名次，避免直接比较原始分数，但仍需选择候选数量、排名平滑常数和权重。任何融合都应拿实际查询验证。

方式	直觉上的优势	常见薄弱处
词法	稳定错误串、专有名词、版本号	同义表达、跨语言、查询词不在文档
训练式向量	语义改写与自然描述	稀有代码、否定差别、领域不匹配
混合	合并候选并利用互补线索	弱组件或不合适权重可能改变正确排序
重排	对少量候选与问题作精细比较	额外延迟、模型误判；补不回漏掉的候选

重排放在哪里

重排是在初步检索后，重新比较少量候选的过程。cross-encoder 可同时读取查询和候选文本打分；也可用明确规则优先同版本文档。它能改善顺序，却不能保证所需资料进入候选集，更不能证明建议适用于本次任务。将召回范围、排序质量与答案忠实度分开检查，才能知道失败发生在哪一层。

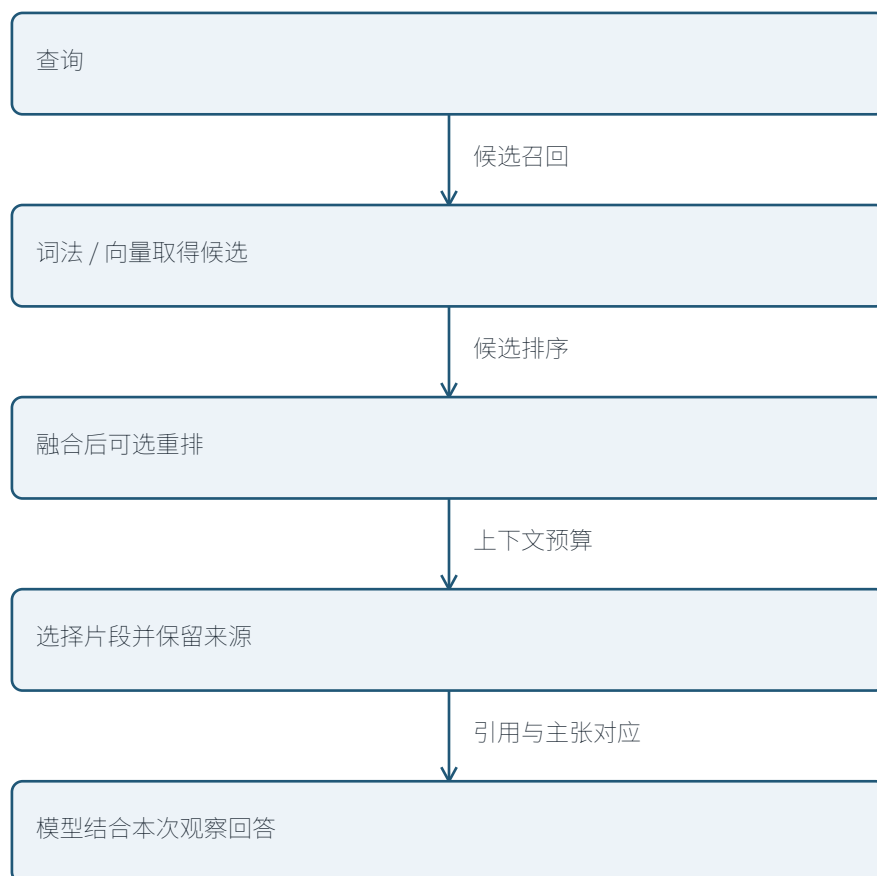


图 3-2 · 一般 RAG 路线；当前 V2 只实现有界词法搜索，未接训练式向量和重排

当前 V2 的 `search_runbooks` 不是 BM25，也没有使用 `src/ailab_ops/rag/` 的 `HybridRetriever`。它用词项集合交集加标题权重检索两篇独立 Runbook。词法/向量/混合/重排是本章解释的设计空间，下一节历史实验展示 retained V1 实现；不得把它们写成 V2 已接入的能力。

3.6 读当前 V2 检索器

`build_runbook_tool` 创建时读取知识目录，保存文件名、标题、正文和词项集合。`_terms` 将文本转小写，用正则提取字母数字或连续中文片段，移除少量英文停用词。它不是中文分词器，不提供翻译或语义展开。编辑 Markdown 后要重建工具实例；这里没有文件变更监听或热更新索引。

源码摘录：正文重叠词项计 1，标题重叠额外计 2；同分按文件名

```
terms = _terms(query)
ranked = []
for filename, title, text, title_terms, body_terms in documents:
    score = len(terms & body_terms) + 2 * len(terms & title_terms)
    if score:
        ranked.append((score, filename, title, text))
ranked.sort(key=lambda row: (-row[0], row[1]))
```

源码定位：`src/ailab_ops/tools/runbooks.py · build_runbook_tool`

`score` 是匹配词项数量的加权值，既不是相似概率，也不是根因置信度。重复输入同一词不会按重复次数加分，因为 `terms` 是集合。工具只保留 `score` 非零候选；`total_matches` 是截取 `top_k` 前的候选数量。

schema 允许 query 长度 1-1000、top_k 为 1-5，默认 top_k=3。通常通过 registry/policy 调用时执行参数检查；前面的直接 fn 示例用于观察内部结果，没有经过授权层。

片段长度与定位

当前每篇文档作为一个候选，不做段落切分。摘录为 text[:4000]，即最多 4000 个 Python 字符，不是 4000 tokens。metadata 保留 doc_id、source、line_start=1 及摘录覆盖的行数 line_end。两个当前 Runbook 均短于上限，因此返回 truncated=False。若正文超长，最后一行可能只返回部分字符，行号表示摘录定位，不应解释为整行完整保留。

源码摘录：摘录上限、来源定位与截断标记

```
excerpt = text[:4000]
metadata = {"doc_id": "runbook:" + Path(filename).stem,
            "source": "data/v2/knowledge/runbooks/" + filename,
            "line_start": 1, "line_end": len(excerpt.splitlines())}
truncated = len(excerpt) < len(text)
```

源码定位：src/ailab_ops/tools/runbooks.py · build_runbook_tool

如果知识扩大，可按标题、症状、辨别方法和操作条件组织 chunk。每个 chunk 应保留父文档、标题路径、版本和位置；重叠可减少边界处丢信息，却会产生重复候选，不能把重复片段当多份独立证据。这些是扩展设计。先用小规模可解释工具建立来源边界，再用实际查询决定是否需要复杂索引。

3.7 V1 基线：弱检索器怎样拖累融合

本节全部属于 V1 基线历史，使用生成日志，且知识与标签共享 faults.yaml。不是 V2 当前指标，不是生产故障准确率，也不是训练式 embedding 的能力比较。保留它是为了理解融合可能失效的机制。

数据和计算方法

保留的数据位于 data/generated/，meta.json 标记 seed=20260929、400 个任务。比较时排除成功、无 root_cause 或 insufficient_evidence 的任务，取 extract_log_evidence 得到的 first_error 前 200 字符，得到 286 个非空查询。它们是生成器产生的日志查询；旧工具打印 real failure queries 的措辞不能当作真实事故来源。

V1 build_knowledge_base 由 playbook 生成 Runbook，再加入平台说明，共 37 篇文档。HybridRetriever 按段落组织 chunk，以 BM25 字段加权作词法侧，以 HashingEmbedder 作向量侧。哈希编码器把词与字符 n-gram 通过固定哈希映射到 256 维并归一化；它没有通过神经网络学习语义。这里的向量检索是一种词袋特征表示实验，不应作为真实 embedding 模型的质量保证。

实验固定 fusion=rrf，扫 lexical_weight；对每个查询取 top-1 文档，检查 scenario metadata 是否等于生成任务的 root_cause。top-1 比例等于正确数除以 286。因标签和知识共享剧本，它适合对照同一素材上的排序变化，却不能检验未知事故泛化。

词法权重 w	top-1 正确数 / 286	top-1 比例
1.00 (词法侧)	236	82.5%

词法权重 w	top-1 正确数 / 286	top-1 比例
0.00 (哈希侧)	83	29.0%
0.50 (等权 RRF)	166	58.0%
0.70	192	67.1%
0.85 (V1 默认权重)	215	75.2%
0.90	219	76.6%

数据重新计算于 2026-10-08、代码基线 64d980e、Python 3.12，固定 PYTHONHASHSEED=0。等权 RRF 比词法侧少 70 个正确 top-1，下降约 24.5 个百分点。移向词法侧能回升，但 w=0.85 仍低于词法侧约 7.3 个百分点。旧章的 57.0% 没有保留同分排序条件，不沿用为固定值。这组测量没有证明 0.85 是全局最优，也没有覆盖释义查询或训练式 embedding 的增益。

复跑时还发现一个历史实现边界：融合候选从 set 遍历，同分文档没有稳定的第二排序键，Python hash seed 改变会影响 top-1。相同数据下，hash seed=1 的等权正确数是 164，hash seed=0 是 166。教材固定环境以复现表格；生产检索应定义同分 tie-break。两种条件下等权融合都明显低于词法侧，教学结论不依赖这两条差异。

源码定位：data/generated/meta.json

源码定位：src/ailab_ops/rag/kb.py · build_knowledge_base

源码定位：src/ailab_ops/rag/embed.py · HashingEmbedder

为什么“只用排名”仍可能错

RRF 的直觉是每个检索器都投一票：第一名贡献较大，后面的贡献较小。当前 V1 加权形式是 $w/(k + \text{词法排名}) + (1-w)/(k + \text{向量排名})$ ，缺席某列表时该项为零， $k=60$ 。它解决量纲问题，却没有辨别高排名是否可靠。若弱组件持续把不相关文档排前面，等权投票就可能压下正确候选。

源码摘录：retained V1 按候选 chunk 的两个排名融合

```
s = 0.0
if cid in lex_rank:
    s += self.lexical_weight / (self.rrf_k + lex_rank[cid])
if cid in dense_rank:
    s += (1.0 - self.lexical_weight) / (self.rrf_k + dense_rank[cid])
fused[cid] = s
```

源码定位：src/ailab_ops/rag/store.py · HybridRetriever.search

正确结论是“融合不会自动胜过组件，应测量组件与组合”；不是“只要两个检索器强弱不同，融合就必然下降”。弱组件如果恰好补足强组件漏掉的查询，仍可提高总体表现。换训练式 embedding、改变领域或增加释义查询后，最优权重可能改变，也可能没有改善，必须重新测量，不能从当前表格推断。

历史复现全部权重的入口是本章内容测试；legacy compare 只比较默认权重下的 rrf 与 linear，并不输出这张权重表。两者用途不同，避免运行一个命令却以为得到同一实验。

课后阅读命令：测试重跑六档；legacy 是显式历史入口

```
PYTHONPATH=src python3 -m pytest tests/test_course_content.py -q \
-k chapter_three_v1_baseline

# 历史工具：仅比较默认权重下两种融合方式
PYTHONHASHSEED=0 PYTHONPATH=src \
python3 -m ailab_ops.cli legacy compare
```

源码定位：tests/test_course_content.py
源码定位：src/ailab_ops/cli.py · cmd_legacy_compare

3.8 证据对象：不把摘录变成无来源文字

检索结果进入上下文后，若只剩自然语言，报告便很难回答“这句话从哪里来”。Evidence将观察内容与读取方式绑定。模型可以解释它，程序可以定位和引用它；证据本身不直接裁定根因。日志、指标和Runbook 工具返回 evidence_items，snapshot 的普通 data 由 orchestrator 标准化为 Evidence。一次读取得到一个证据对象，不等于里面只包含一行原文。

字段	含义	案例中的阅读方式
evidence_id	稳定来源内容身份	报告引用 ev-...；不是随机编号
source_tool / arguments	由什么调用取得	get_case_logs / case_id；search_runbooks / query
summary / excerpt	摘要与保留摘录	解释要能回到 excerpt，不只读摘要
observed_time_range	观察起止时间，允许 None	日志 01:00:00–01:11:03 UTC；手册无事件时间
truncated	工具是否裁切返回内容	False 不代表原始遥测完整
relation / hypothesis_id	关系及可选假设标记	related / None 为默认值
metadata	来源定位与采集范围	手册路径/行号；日志 complete、available_ranks

源码摘录：Evidence 保存观察和来源，不带根因真相字段

```
class Evidence:
    source_tool: str
    arguments: dict[str, Any]
    summary: str
    excerpt: str
    evidence_id: str = ""
    observed_time_range: tuple[str, str] | None = None
    truncated: bool = False
    relation: EvidenceRelation = EvidenceRelation.RELATED
    hypothesis_id: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
```

源码定位：src/ailab_ops/evidence/models.py · Evidence

稳定 ID 与去重的实际边界

EvidenceStore.add 将 source_tool、arguments、excerpt 规范化为 JSON，再计算 SHA-256，加 ev- 前缀。同一来源、参数和摘录得到相同 ID；其中任一项改变都会形成新的身份。ID 很长，阅读时可用短前缀帮助区分，机器引用必须保存完整值。哈希表达内容身份，不提供来源真实性或签名认证。

阅读示意：原字典展开分行，字段与 ID 计算不变

```
source = json.dumps(
    {
        "source_tool": evidence.source_tool,
        "arguments": evidence.arguments,
        "excerpt": evidence.excerpt,
    },
    sort_keys=True, separators=(",", ":"), ensure_ascii=False, allow_nan=False,
)
evidence_id = "ev-" + sha256(source.encode("utf-8")).hexdigest()
```

源码定位：src/ailab_ops/evidence/store.py · EvidenceStore.add

摘要、metadata、时间范围和关系不参与当前哈希。存储保留首次插入，并返回独立副本；读取后修改副本不会改掉保存的证据。同一内容后续补 metadata 不会自动更新第一次记录。向真实平台扩展时应设计版本或采集身份，避免把重新采集、来源变化和-content 去重混为一谈。

3.9 完整性、截断与假设关系

没有看到不等于没有发生

truncated 回答“工具有没有剪掉返回内容”。telemetry.complete / available 回答“源头是否完整、是否采集过”。某工具把全部保留的两行尾日志交回来，truncated=False，但此前半小时日志仍可能丢失。若只检查截断标记，就会把缺失资料误当成完整阴性证据。

案例	工具裁切	采集情况	可推出的结论
GPU assert 日志	truncated=False	complete=True；四 rank 可见	可比较首个已观察错误；仍是精选素材
证据不足日志	truncated=False	complete=False；仅 launcher tail	尾日志无 OOM 不能排除 OOM
证据不足指标	truncated=False	available=False；未启用 exporter	没有指标，不能宣称资源健康

当前 observe 返回全部保留行，把 telemetry 和 source_note 放入 metadata。GPU assert 的 source_note 仍说明没有 sample payload 或 kernel 同步 trace，所以完整的保留日志不等于取得所有可想象的资料。30 秒采样的内存序列只能证明这些样本未达容量，不能排除两次采样之间的一瞬间。“在保留样本中”这样的限定是判断的实质部分。

源码定位：src/ailab_ops/tools/cases.py · build_case_registry

支持、反驳与相关不是同一种关系

EvidenceRelation 允许 supports、contradicts、related。Hypothesis 另存 supporting_evidence_ids与 contradicting_evidence_ids：一个候选解释要说明哪些观察增加可信度，哪些与它冲突。默认读到的 Evidence 只是 related，没有自动转成 supports；模型提交假设时引用 ID，控制层检查 ID 属于当前会话，却不自动推断语义关系。

rank 2 早于 peer timeout 的断言支持本地异常先发生。低内存样本可削弱“持续资源耗尽”，却不能全面反驳“瞬时显存峰值”。Runbook 的一般说明与多个假设都可能有关，不能把相关性当独立实测支持。当前 Evidence.relation / hypothesis_id 与 Hypothesis 两个引用列表没有自动双向同步；后者是模型提交的关系表达，前者主要是证据元数据预留。

源码定位：src/ailab_ops/investigation/models.py · Hypothesis

源码定位：src/ailab_ops/investigation/orchestrator.py · _control

3.10 主张—证据校验：引用合法仍有边界

把重要结论拆成可核查的句子

Claim 由 text、evidence_ids 和 material 组成。material=True 表示实质主张，需要至少一个引用。“rank 2 的索引断言先于 peer watchdog”可回到日志时间线；“保留内存样本未显示容量耗尽”可回到指标和容量。它们比一整段“因此有问题，建议重启”容易核查。material 标记仍由模型填写，程序不会自动把关键自然语言句子识别为主张。

阅读示意：原调用展开分行，实质主张缺引用时形成错误

```
for claim_index, claim in enumerate(report.claims):
    citations_path = f"claims[{claim_index}].evidence_ids"
    if claim.material and not claim.evidence_ids:
        issues.append(
            ValidationIssue(
                "missing_citation", citations_path,
                "Material claim requires at least one evidence citation.",
            )
        )
```

源码定位：src/ailab_ops/evidence/validation.py · validate_report

检查	当前报错	防止的错误
实质主张是否引用	missing_citation	关键 Claim 无来源
引用是否存在	unknown_evidence	编出不存在的 ev ID
同一主张重复引用	duplicate_citation	同一 ID 重复凑数量
至少一个实质主张	missing_material_claim	全部 material=False 绕过基本要求
当前会话观察	out_of_session_evidence	引用其他会话来源

前三项由 validate_report 检查，后两项由 orchestrator._report 补充。程序进入 validating，问题回喂给模型，允许一次报告修正；再次失败则 stopped/report_validation_failed。通过时保存 report 并 completed。引用检查让模型不能任意制造可追溯性，也让报告发布拥有确定的结构要求。

源码定位：src/ailab_ops/investigation/orchestrator.py · _report

引用存在但结论仍错误的反例

假设报告写“网络接口故障已确定”，却引用 GPU assert 案例的有效日志 ID。这个 ID 的确存在，没重复，也属于本会话。结构校验可能通过；它不读取自然语言与摘录，推断二者是否在语义上蕴含。当前代码没有语义 entailment judge，也不自动检查报告是否限定 truncated 或缺失 telemetry。不能将 citation_validity 或结构通过称为证据忠实度已被证明。

当前主张—证据关联的保证是“每个 material Claim 带可定位且属于会话的引用”。语义支持、因果充分性和结论覆盖仍需在线评测、人工抽查或未来独立验证器。验证器也可能出错，应独立评测，不能用另一个模型的肯定替代原始证据。

默认 authored replay 给两个主张都附四份证据，包括 snapshot、logs、metrics 和 Runbook，便于端到端演示，却是较粗的引用粒度。“每条引用都合法”不能推导为“每份资料都独立支持每个句子”。部署前可让主张仅引用有关证据，再细化到日志行、指标区间和文档段落；这是改善项，不是本章已经新增的实现。

3.11 证据不足：带事实与缺口结束

case-insufficient-evidence 只保留两条 launcher 尾日志：Received SIGTERM 和 exit 143。早期 worker stderr、scheduler termination events、信号发送者或容器退出原因没有保留，资源指标未采集。SIGTERM 和 143 描述终止过程，没有告诉我们谁发信号、为什么发信号。直接写成内存不足、网络中断或节点故障，都是跨过了证据缺口。

回放仍调用 search_runbooks；知识说明 timeout 或终止尾日志不足以定位起因。检索返回文档，不会凭空补齐本次任务的源头证据。最终 report.root_cause=insufficient_evidence、confidence=0.0，保留已观察事实，列出四项未知信息，建议获取 earlier logs 和 scheduler termination events。

源码定位：data/v2/cases/case-insufficient-evidence.json

这里的拒答是拒绝确定无依据的根因，不是拒绝提供所有帮助。读者应看到三部分：能确认的终止事实、无法区分候选解释的缺口、下一步需要的资料。调查可以 completed，因为一份有效的“不足以归因”报告已经完成。completed 不表示已找出原因，更不表示故障修复。

报告部分	当前回放内容	为什么有用
结论	insufficient_evidence; confidence=0.0	避免把终止码当根因
事实	SIGTERM / exit 143；早期记录与指标缺失	说明资料及覆盖范围
未知项	worker stderr、调度事件、信号/容器原因、资源/节点遥测	下一次采集要补什么
建议	先索取早期日志与调度终止事件	提供调查方向

提示词要求状态不确定性，控制层却不会从缺失 telemetry 自动强制 insufficient_evidence。结构合法但自信归因的在线回答仍可能发布，必须通过拒答评测和案例审查发现。本次回放拒答是作者写好的响应，只能证明它经过相同引用管线可以呈现，不能证明在线模型的拒答能力。

源码定位：src/ailab_ops/investigation/prompts.py

3.12 运行证据与反例检查

两次可复跑调查

读取已有教学响应：两个命令均生成 JSON 报告，退出码 0

```
PYTHONPATH=src python3 -m ailab_ops.cli investigate \
  --case case-gpu-assert --mode replay
PYTHONPATH=src python3 -m ailab_ops.cli investigate \
  --case case-insufficient-evidence --mode replay
```

观测	GPU assert	证据不足
mode / phase	replay / completed	replay / completed
模型决策步 / token usage	5 / 1000	5 / 1000

观测	GPU assert	证据不足
证据数 / material Claim 数	4 / 2	4 / 2
root_cause	gpu_device_assert	insufficient_evidence
confidence / unknowns 数	0.9 / 1	0.0 / 4

两次证据来源都依次为 get_case_snapshot、get_case_logs、get_case_metrics、search_runbooks。模型网关每轮使用 data/v2/replays/investigations.jsonl 中 authored 请求/响应，工具照常执行。证据 ID、行号、采集范围与检索得分来自当前代码，根因文本与 confidence 来自预置响应。5 步指模型决策次数，1000 tokens 是记录中的 usage 累计，不是真实 API 成本或在线耗时。

源码定位：data/v2/replays/investigations.jsonl
源码定位：src/ailab_ops/models/replay.py · ReplayModelGateway

阅读时主动寻找这些反例

错误做法	会误导什么	应怎样判断
检索空泛问题	无结果变成无知识可用	先读观察，检查术语与语言
检索“坏网卡修复”后归因	假设通过查询变成结论	保留观察，与竞争解释比较
文档第一名就是根因	排序分数变成诊断概率	回到本次日志和指标验证
truncated=False 就完整	缺失记录当作阴性证据	同时读 telemetry、source_note
引用多就证据充分	宽泛引用掩盖语义跳跃	逐主张核查内容与适用范围
V1 82.5% 当 V2 准确率	共享剧本排序当模型泛化	核查模式、数据、标签、命令

这些检查也适用于客服或代码问答：文档有版本与条件，当前问题有独立事实，更换向量库不能抹平边界。评价 RAG 时，既要知道所需文档有没有进候选，也要知道输出有没有忠实使用它，以及结论是否超出资料证明范围。

3.13 本章总结与课后阅读

RAG 提供外部资料；证据链连接本次观察、知识来源和报告主张。观察后检索提升查询信息量，独立知识/标签目录避免运行时直接泄漏答案。词法、向量、融合和重排各有薄弱处；V1 基线展示弱组件可能拉低排序，却没有给 V2 或真正 embedding 下定论。

稳定 ID 帮助定位，不证明原始信息真实；truncated 和 telemetry 回答不同问题。当前控制层检查 material Claim 的引用存在、重复及会话范围，不证明语义支持。资料缺失时应输出有边界的事实和补证方向，在线模型能否做到仍需评测。

一条建议的源码阅读路线

1. 从 tools/runbooks.py 读 _terms 与 build_runbook_tool，手算 query score，再比较 QUERY_OBSERVATIONS。

2. 看 tools/cases.py 与案例 telemetry，区分保留行、源头缺失和工具裁切。
3. 读 evidence/models.py、store.py，找出参与 ID 的字段和首次插入语义。
4. 读 evidence/validation.py 与 orchestrator._report，列出能拦住和拦不住的错误。
5. 对照两个回放的 claims / unknowns，再看 V1 rag/store.py，区分当前工具与历史实验。

源码定位：src/ailab_ops/evidence/validation.py · validate_report

课后模仿方向与思考

课后先复跑已有命令，任选一条 Claim，将 evidence_ids 对照 evidence 数组，找到有关原文。用纸面检查表记录：支持哪部分结论、覆盖什么时间、还有哪些缺口。再尝试为熟悉的 Python 报错写简短 Runbook，包含症状、辨别条件、补证步骤和适用边界；不要把测试题的答案标签放进去。

1. 检索第一名 score 从 4 变 8，为什么不能直接把根因 confidence 提高一倍？
2. 工具返回空 rows、truncated=False、complete=False，哪些否定性结论不成立？
3. 怎样证明 embedding 帮助中文释义查询，同时没降低精确错误串排序？需要怎样分组数据？
4. material Claim 引用合法日志却写错原因，当前校验怎么处理？如何设计独立语义审查？
5. 两篇文档矛盾时，需要哪些版本和来源信息才能决定适用性？
6. 证据不足报告为何可以 completed？它和 stopped 的预算中断有什么不同？

章节关联：第 4 课 · 下一章将带引用建议接到策略与审批，解释建议、批准与执行的边界。